Personalized Technical Interview Preparation in Higher Education (SmartInterview-G1392)

Project Team ID: Team G-1392

1. Shamakura Saiteja Goud - 24BD1A665K

2. Vasireddy Vignesh Reddy - 24BD1A665X

3. Thodsam Srujan - 24BD1A665R

4. Aelugu Ranith Kumar - 25BD5A6615

5. Konduri Abhiram - 25BD5A6620

Problem Statement:

In modern technical recruitment, graduating engineering students face severe preparation barriers including static question banks, anxiety during live technical interviews, lack of personalized feedback, and complete disconnect between university curricula, candidate resumes, and dynamic industry job descriptions. Existing mock interview platforms rely on rigid question sets or ungrounded generative AI bots that hallucinate questions, exhibit significant conversational latency, and lack systematic evaluation methodologies.

When candidates prepare for specific software engineering roles, their target job descriptions require specific technical skillsets—spanning Data Structures, System Design, Operating Systems, Database Management Systems, Computer Networks, and Machine Learning. Static platforms fail to compare candidate resumes against targeted job descriptions to identify exact role overlap and critical skill deficiencies.

Furthermore, traditional evaluation mechanisms fail to measure cognitive depth. When an interview question is presented, candidate responses must be evaluated across multiple technical dimensions—including technical correctness, completeness, conceptual coverage, semantic relevance, and communication clarity. Without dynamic difficulty scaling based on validated educational frameworks, candidates cannot assess their true interview readiness or receive actionable remediation paths.

To overcome these limitations, SmartInterview provides an AI-powered, RAG-grounded adaptive technical mock interview platform. By integrating dense vector retrieval with Bloom's Taxonomy cognitive progression, multi-signal response evaluation, in-memory neural text-to-speech, and automatic speech recognition transcription, the system delivers an authentic, personalized, low-latency interview preparation experience.

Software Requirement Specification

For

Personalized Technical Interview Preparation in Higher Education

Prepared by:

1. Shamakura Saiteja Goud - 24BD1A665K

2. Vasireddy Vignesh Reddy - 24BD1A665X

3. Thodsam Srujan - 24BD1A665R

4. Aelugu Ranith Kumar - 25BD5A6615

5. Konduri Abhiram - 25BD5A6620

Project Mentor: Dr. TVG Sridevi

Department of Computer Science and Engineering

Keshav Memorial Institute of Technology (KMIT)

September 2026

Table of Contents

Table of Contents **3**
Revision History **4**
1. Introduction **5**
1.1 Purpose 5
1.2 Document Conventions 5
1.3 Intended Audience and Reading Suggestions 5
1.4 Product Scope 5
1.5 References 6
2. Overall Description **7**
2.1 Product Perspective 7
2.2 Product Functions 7
2.3 Operating Environment 8
2.4 User Characteristics 8
2.5 Design and Implementation Constraints 8
2.6 Assumptions and Dependencies 8
3. Specific Requirements **9**
3.1 Functional Requirements 9
3.2 Non-Functional Requirements 11
Software Development Life Cycle (SDLC) **13**
Stages of SDLC 13
SDLC Models & Methodology Selection 14
Literature Survey **15**
4. System Designing & UML Diagrams **17**
Overview of UML & Conceptual Model 17
Structural vs. Behavioral Diagrams 17
4.1 Use Case Diagram & Template 18
4.2 Sequence Diagrams 20
4.3 Collaboration Diagram 21
4.4 Class Diagram 22
4.5 Statechart Diagram 23
4.6 Activity Diagram 24
4.7 Package Diagram 25
4.8 Component Diagram 26
4.9 Deployment Diagram 27
4.10 Entity Relationship Diagram (ERD) 27
4.11 Data Flow Diagrams (Level 0 and Level 1) 28
5. Implementation **29**
6. Testing (Validation, Verification & Test Cases) **30**
7. Conclusion **31**

Revision History

Name	Date	Reason for changes	Version
Week-1	12-07-2026	Problem Statement, Team Constitution & Feasibility	1.0
Week-2	19-07-2026	Literature Review & Research Gap Analysis	2.0
Week-3	26-07-2026	Functional & Non-Functional Requirements (SRS Drafting)	3.0
Week-4	02-08-2026	SDLC Methodology Selection & Academic Template Align	4.0
Week-5	09-08-2026	Use Case Modeling & Use Case Template Specification	5.0
Week-6	16-08-2026	Sequential and Collaboration Diagrams Specification	6.0
Week-7	23-08-2026	Class Diagram & Relational Entity-Relationship Design	7.0
Week-8	30-08-2026	Statechart and Activity Diagrams Modeling	8.0
Week-9	06-09-2026	Package, Component, and Deployment Architecture	9.0
Week-10	13-09-2026	Data Flow Modeling, Testing Cases & Final SRS Approval	1.0 (Final)

1. Introduction

The Software Requirement Specification (SRS) is designed to document and describe the technical agreement between the candidate, project stakeholders, academic evaluators, and system developers regarding the specification of the SmartInterview software platform.

This documentation provides a comprehensive specification of system functional behavior, performance parameters, and architectural constraints. It serves as the authoritative technical baseline for implementation, system testing, architectural review, and deployment verification.

1.1 Purpose:

This document specifies the software requirements for Personalized Technical Interview Preparation in Higher Education. The system automates the technical interview preparation lifecycle for engineering candidates. It provides personalized interview sessions grounded in domain-specific computer science knowledge, adjusts question difficulty dynamically based on candidate performance, delivers spoken voice output, transcribes verbal answers, and produces comprehensive analytical evaluation reports.

1.2 Document Convention:

The document conforms to academic software engineering documentation standards and institutional guidelines. The formatting hierarchy is structured as follows:

- Section Headings (H1): Font-Size: 16 pt, Bold, Font: Times New Roman
- Subheadings (H2): Font-Size: 14 pt, Bold, Font: Times New Roman
- Sub-subheadings (H3): Font-Size: 12.5 pt, Bold, Font: Times New Roman
- Body Content: Font-Size: 12 pt, Line-Spacing: 1.15, Font: Times New Roman
- Requirement Identifiers: Each functional requirement is uniquely designated as RI.01.XX and priority levels range from 1 (Critical) to 4 (Informational).

1.3 Intended Audience and Reading Suggestions:

This document is intended for project mentors, faculty evaluators, system developers, QA engineers, and engineering candidates. Developers should focus on Sections 2, 3, and 4 for system architecture, API specifications, and UML models. Academic evaluators and mentors should review the Problem Statement, SDLC methodology, Literature Survey, and Requirements Specification to assess project rigor and architectural soundness.

1.4 Product Scope:

SmartInterview is an intelligent web-based application designed to bridge the preparation gap between university education and software industry hiring expectations. The platform provides continuous 24x7 self-directed interview practice. Unlike generic chat interfaces, SmartInterview grounds interview content in an authoritative local vector store of 402 curated computer science

topics across 8 technical domains (DSA, DBMS, OS, CN, System Design, OOP, Design Patterns, and ML/DL).

The system analyzes candidate resumes and target job descriptions to compute skill overlap and role deficiencies, conducts dynamic multi-turn technical interviews across Bloom's Taxonomy cognitive tiers, evaluates answers across 5 technical and semantic criteria, adjusts subsequent question difficulty, and provides spoken voice delivery alongside a downloadable performance evaluation report.

1.5 References:

[1] A. Wahid, A. Jha, M. Munshi, S. Sonwane, and A. Chakrawarti, 'AI Mock Interview: An Intelligent Voice-Driven Interview Simulation System using Gemini AI and Whisper,' International Journal of Engineering Research & Technology (IJERT), vol. 15, no. 03, pp. 214-221, Mar. 2026. (Base Paper 1)

[2] M. K. Nagarajan, A. Kumar, I. V. S. Vignesh, A. Nehal, and M. Musthafa, 'Development of an Autonomous AI Interview Engine Using Sentence-BERT and Large Language Models for End-to-End Candidate Evaluation,' Indian Journal of Modern Research and Reviews (IJMRR), vol. 4, no. 2, pp. 33-40, Feb. 2026. (Base Paper 2)

[3] P. Lewis et al., 'Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,' NeurIPS, 2020.

[4] L. W. Anderson and D. R. Krathwohl, A Taxonomy for Learning, Teaching, and Assessing: A Revision of Bloom's Taxonomy of Educational Objectives. Longman, 2001.

[5] A. Radford et al., 'Robust Speech Recognition via Large-Scale Weak Supervision,' ICML, 2023.

[6] Groq Inc., 'Language Processing Unit (LPU) Hardware Architecture Overview,' Whitepaper, 2024.

[7] FastAPI Framework Documentation, Version 0.115.0, https://fastapi.tiangolo.com

[8] ChromaDB Vector Database Technical Reference, Version 0.6.3, https://docs.trychroma.com

2. Overall Description

2.1 Product Perspective

SmartInterview is an independent, self-contained three-tier web application integrated with local vector retrieval and high-speed cloud inference. The presentation tier consists of a responsive React single-page application; the application tier is powered by a FastAPI backend running under Uvicorn ASGI; the persistence tier utilizes a relational MySQL 8.0 database for user accounts and session records, and an embedded ChromaDB 0.6.3 instance for vector embeddings.

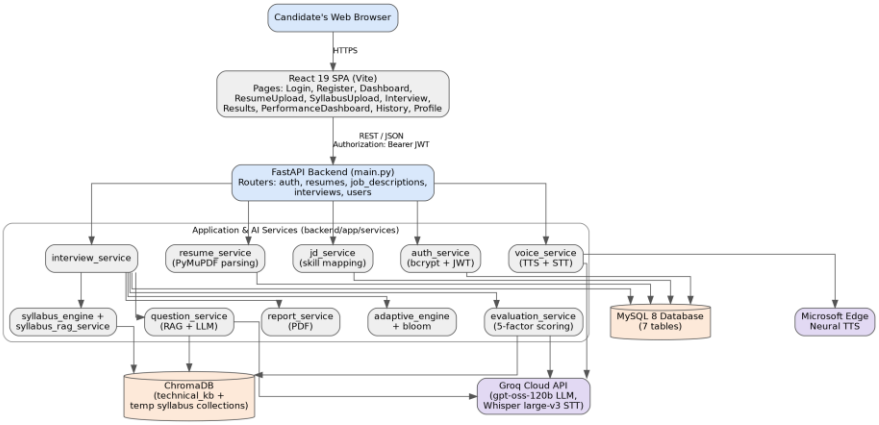


Figure 2.1 — Layered System Architecture and Context of SmartInterview

2.2 Product Functions

2.2.1 Candidates / Users

- User registration, authentication with JWT, and profile management.
- Resume upload (PDF) with automated section parsing and skill extraction.
- Job description upload (PDF) with Group A (Role Overlap) and Group B (Role Gap) mapping.
- Syllabus mode upload (PDF/DOCX/TXT) creating isolated temporary vector collections.
- Interactive technical mock interviews with real-time text and speech input options.
- Instant multi-signal performance scoring and cognitive depth progression.
- Comprehensive performance dashboard, session history, and downloadable PDF report.

2.2.2 System / Adaptive RAG Engine

- Deterministic skill gap calculation between candidate resumes and job descriptions.
- Domain-filtered semantic retrieval of top-3 context chunks from 402 curated computer science topics.
- Bloom-levelled single-question generation powered by Groq LLM (openai/gpt-oss-120b).
- Deterministic adaptive progression adjusting difficulty and cognitive depth after each response.
- In-memory neural text-to-speech question synthesis via Microsoft edge-tts.
- Speech-to-text audio transcription via Groq Whisper API (whisper-large-v3).

2.3 Operating Environment

The system operates smoothly across the following hardware and software configuration:

- Client Operating System: Windows 10/11, macOS, or modern Linux distributions.
- Client Hardware: Commodity desktop or laptop with at least 4 GB RAM and working microphone interface.
- Client Web Browser: Google Chrome 90+, Microsoft Edge 90+, Mozilla Firefox 88+, or Safari 14+ with JavaScript enabled.
- Server Runtime: Python 3.10+ (FastAPI 0.115.0, Uvicorn 0.30.0, SQLAlchemy 2.0.35).
- Frontend Runtime: Node.js 18+ (React 18 / 19, Vite build tool, Tailwind CSS).
- Database Engines: MySQL Server 8.0+ (Port 3306) and ChromaDB 0.6.3 (Local embedded SQLite/HNSW store).
- External AI Cloud: Groq Cloud API (valid GROQ_API_KEY) and Microsoft edge-tts.

2.4 User Characteristics

The primary users of SmartInterview are undergraduate engineering students, job seekers, and software engineering candidates preparing for technical campus placements and industry interviews. Users are assumed to possess basic familiarity with web browsers and file upload workflows; no advanced programming knowledge is required to operate the interview interface.

2.5 Design and Implementation Constraints

- An active internet connection is mandatory for cloud LLM generation (Groq API) and speech transcription.
- A valid GROQ_API_KEY must be configured in the project .env configuration file.
- Document uploads (Resume, Job Description, Syllabus) are strictly restricted to 5.0 MB per document.
- Stateful candidate sessions rely on signed JWT Bearer tokens passed via HTTP Authorization headers.
- The permanent RAG knowledge base is constrained to eight core computer science domains (cn, dbms, design-patterns, dsa, ml-dl, oop, os, system-design).

2.6 Assumptions and Dependencies

- Candidate resumes, job descriptions, and course syllabi are provided in English in supported formats (PDF, DOCX, TXT).
- The client browser grants microphone permissions when voice answering mode is selected.
- The MySQL database schema in backend/setup_database.sql has been initialized prior to starting the application.
- Third-party libraries: FastAPI, PyMuPDF, SentenceTransformers, ChromaDB, edge-tts, ReportLab, and Groq SDK.

3. Specific Requirements

3.1 Functional Requirements

The functional requirements specify the exact behavioral responses and operations of SmartInterview across all candidate modules, formulated according to the requirement specification template:

Requirement ID : RI.01.01
Title : Candidate Account Registration
Description : The system shall allow new candidates to register using email, full name, and password. The password must be hashed using bcrypt (12 rounds) with 72-byte truncation protection before database storage in MySQL users table.
Priority : 1

Requirement ID : RI.01.02
Title : Candidate Authentication & JWT Issuance
Description : The system shall authenticate existing candidates against stored bcrypt hashes and issue an HS256-signed JSON Web Token (JWT) valid for 24 hours (1440 minutes) upon successful credential verification.
Priority : 1

Requirement ID : RI.01.03
Title : Resume Ingestion & PDF Parsing
Description : The system shall accept candidate resume uploads (PDF format, <= 5 MB), validate file headers, parse text streams using PyMuPDF (fitz), and extract technical skills, projects, and education history.
Priority : 1

Requirement ID : RI.01.04
Title : Job Description Upload & Role Extraction
Description : The system shall parse target job descriptions (PDF <= 5 MB), extracting required role qualifications, mandatory skills, and technical responsibilities to guide personalized interview staging.
Priority : 2

Requirement ID : RI.01.05
Title : Skill Overlap and Gap Mapping
Description : The system shall compute the set intersection and set difference between candidate resume skills and job description skills, partitioning them into Group A (Role Overlap) and Group B (Role Gaps).
Priority : 2

Requirement ID : RI.01.06
Title : Syllabus Mode Ingestion & Vector Staging
Description : The system shall allow candidates to upload course syllabi (PDF, DOCX, TXT <= 5 MB), generate 384-dimensional embeddings via SentenceTransformers all-MiniLM-L6-v2, and stage an isolated temporary ChromaDB collection temp_syllabus_<id>.
Priority : 3

Requirement ID : RI.01.07
Title : RAG-Grounded Bloom-Adaptive Question Generation
Description : The system shall dynamically retrieve top-3 relevant context chunks from ChromaDB for the target skill, assemble prompt instructions incorporating Bloom's Taxonomy cognitive verbs, and invoke Groq LLM (openai/gpt-oss-120b) to generate exactly one contextual technical question.
Priority : 1

Requirement ID : RI.01.08
Title : Multi-Signal 5-Factor Answer Evaluation

Description : The system shall evaluate candidate answers across five dimensions: Technical Correctness (30%), Completeness (20%), Relevance (20%), Semantic Similarity via SentenceTransformers (15%), and Concept Coverage (15%), producing an overall score on a 0-100 scale.
Priority : 1

Requirement ID : RI.01.09
Title : Deterministic Adaptive State Progression
Description : The system shall deterministically adjust Bloom's cognitive level and question difficulty: Score >= 80% advances Bloom level (+1) and scales difficulty; 50% <= Score < 80% maintains level; Score < 50% regresses Bloom level (-1).
Priority : 1

Requirement ID : RI.01.10
Title : Neural Text-to-Speech Question Streaming
Description : The system shall synthesize neural voice audio of the interview question using Microsoft edge-tts (en-US-ChristopherNeural) and stream audio/mpeg directly in-memory to the client without creating temporary disk files.
Priority : 3

Requirement ID : RI.01.11
Title : Whisper Speech-to-Text Answer Transcription
Description : The system shall accept recorded candidate microphone audio (WebM or WAV), dispatch the audio stream to Groq Whisper (whisper-large-v3), and return verbatim transcribed text for automated evaluation.
Priority : 3

Requirement ID : RI.01.12
Title : Downloadable PDF Interview Report
Description : The system shall generate a server-side formatted PDF report using ReportLab upon session completion, containing candidate details, score breakdown, radar chart of Bloom levels, and detailed question-by-question analysis.
Priority : 2

Requirement ID : RI.01.13
Title : Performance Analytics Dashboard
Description : The system shall aggregate historical candidate session data across all completed interviews, visualizing skill proficiency across computer science domains and cognitive depth distributions.
Priority : 2

3.2 Non-Functional Requirements

3.2.1 Performance Requirements

- Semantic Search Latency: Local ChromaDB top-3 context retrieval shall complete in <= 100 ms on CPU.
- Generation Latency: Groq Cloud LLM question formulation and critique generation shall complete in <= 2.5 seconds.

• TTS Streaming Latency: Neural voice streaming via edge-tts shall commence playback in <= 1.2 seconds.

• Database Operations: Relational session commits and state persistence in MySQL shall execute in <= 50 ms.

3.2.2 Safety Requirements

The software operates purely as an instructional web application and shall not cause physical harm or hardware damage. In-memory temporary audio buffers are wiped immediately following transcription to guarantee zero disk leakage.

3.2.3 Security Requirements

• Authentication: User access is guarded by HS256-signed JSON Web Tokens (JWT) with 24-hour expiration.

• Password Storage: User passwords are encrypted using bcrypt with adaptive salt work factor of 12 rounds.

• File Upload Sanitization: Uploaded resumes and job descriptions undergo MIME-type header validation and file size verification (<= 5 MB limit).

• Environment Secrets: Third-party API keys (GROQ_API_KEY, SECRET_KEY) are decoupled from codebase via .env configurations.

3.2.4 Error Handling

• Network Disruption: If Groq Cloud LLM becomes unreachable, the system executes automatic retry with fallback to openai/gpt-oss-20b before surfacing a structured error to the client.

• Microphone Faults: If browser microphone permission is denied or audio capture fails, the system provides immediate fallback to text-based typing without session invalidation.

Software Development Life Cycle (SDLC)

The Software Development Life Cycle (SDLC) is a structured process used by software engineering teams to design, develop, test, and maintain high-quality software products. The SDLC aims to produce software that meets customer expectations, satisfies formal requirements, and achieves completion within established timeline and resource constraints.

- SDLC is the acronym for Software Development Life Cycle.
- It is also formally designated as the Software Development Process.
- SDLC establishes an operational framework defining specific tasks performed at each phase of the engineering lifecycle.
- ISO/IEC 12207 is the international standard governing software life-cycle processes, defining requirements for software development and maintenance.

What is SDLC?

SDLC is a disciplined process followed in software organizations. It consists of a detailed roadmap describing how to develop, maintain, replace, alter, and enhance software systems. A typical Software Development Life Cycle consists of the following 6 sequential stages:

Stage 1: Planning and Requirement Analysis

Requirement analysis is the most fundamental stage of SDLC. In SmartInterview, requirements were gathered by analyzing candidate interview bottlenecks, academic syllabus standards, and technical job descriptions. Operational and technical feasibility studies confirmed that combining open-weights embeddings (SentenceTransformers) with Groq cloud inference satisfies sub-second latency and zero-cost local retrieval constraints.

Stage 2: Defining Requirements

Requirements are formally defined and documented in the Software Requirement Specification (SRS). All functional capabilities (RI.01.01 through RI.01.13), non-functional constraints, and performance benchmarks are detailed and approved by faculty mentors and academic evaluators.

Stage 3: Designing the Product Architecture

System architects construct the Design Document Specification (DDS). In SmartInterview, multiple architectural approaches were evaluated. A three-tier decoupled client-server architecture was selected, incorporating FastAPI for asynchronous services, ChromaDB for local vector embeddings, MySQL for ACID persistence, and React for the single-page application.

Stage 4: Building or Developing the Product

In this stage, actual source code is written following structured modular programming conventions. The SmartInterview backend was developed in Python 3.10+ using FastAPI and SQLAlchemy; the frontend was built in React 18 and Vite. Modular microservices manage authentication, RAG retrieval, LLM prompt formatting, adaptive Bloom progression, and TTS/STT processing.

Stage 5: Testing the Product

This stage involves rigorous software testing where software defects are tracked, fixed, and retested against SRS specifications. SmartInterview underwent unit testing for JWT authorization, integration testing for ChromaDB retrieval and Groq LLM pipelines, and end-to-end user acceptance testing across mock interview workflows.

Stage 6: Deployment in the Market and Maintenance

Once tested, the software is deployed to staging and production environments. SmartInterview is staged using Uvicorn ASGI behind Nginx reverse proxy with automated MySQL schema migration scripts, complemented by monitoring logs for API latency and token consumption.

SDLC Models

Various software development life cycle models are established in the software industry, each following a unique sequence of phases:

- Waterfall Model: Linear-sequential phase progression with rigid stage sign-offs.
- Iterative Model: Incremental development through repeated cycles (iterations) of small feature sets.
- Spiral Model: Risk-driven model combining iterative development with systematic risk assessment.
- V-Model (Verification and Validation): Parallel execution of development and corresponding testing phases.
- Big Bang Model: Unplanned model with minimal planning, suitable only for very small academic exercises.
- Agile Methodology: Iterative, sprint-based framework emphasizing continuous integration, modular delivery, and adaptability.

Project SDLC Methodology Selection: SmartInterview was executed using the Agile / Iterative Methodology, adhering to a 14-week incremental development plan (from Week 1 Problem Statement through Week 14 Final Deployment). This allowed weekly sprint deliverables, rapid feedback from project mentors, modular integration of the RAG pipeline, and continuous validation of adaptive learning rules.

Literature Survey

A literature survey in a project report examines published research, analyses, and benchmarks in the field of interest, contextualizing the project within the state of the art and establishing the foundational research gap.

1. Voice-Interactive Interview Simulation using Gemini AI & Whisper (Wahid et al., IJERT 2026 - Base Paper 1)

Wahid et al. introduced an intelligent voice-interactive interview simulation platform leveraging Google Gemini LLM and OpenAI Whisper for speech transcription. While demonstrating the psychological value of voice interaction in reducing interview anxiety, the system lacks dynamic knowledge grounding (resulting in factual hallucinations in advanced topics) and operates on a static prompt sequence without formal cognitive scaffolding or difficulty calibration.

2. Autonomous AI Interview Engine Using Sentence-BERT (Nagarajan et al., IJMRR 2026 - Base Paper 2)

Nagarajan et al. proposed an end-to-end interview pipeline utilizing Sentence-BERT (SBERT) dense semantic embeddings for resume-job description alignment. Their work established that dense embeddings significantly outperform lexical keyword matching for candidate evaluation. However, SBERT utility was restricted exclusively to initial resume screening, leaving interview questioning and response scoring to ungrounded LLM prompts without curricular syllabus ingestion or cognitive depth adaptation.

3. A Revision of Bloom's Taxonomy of Educational Objectives (Anderson & Krathwohl, 2001)

Anderson and Krathwohl revised Bloom's cognitive taxonomy into six hierarchical tiers: Remember, Understand, Apply, Analyze, Evaluate, and Create. SmartInterview incorporates this cognitive taxonomy as a deterministic state machine, ensuring technical questions systematically assess cognitive depth rather than random syntax recall.

4. Robust Speech Recognition via Large-Scale Weak Supervision (Radford et al., 2023 - OpenAI Whisper)

Whisper demonstrated state-of-the-art zero-shot speech-to-text accuracy across diverse accents, background noise, and technical jargon. SmartInterview integrates Groq-hosted Whisper (whisper-large-v3) to deliver real-time candidate speech transcription without local GPU requirements.

5. Language Processing Unit (LPU) Tensor Processing Architecture (Groq Inc., 2024)

Groq's deterministic LPU architecture provides sequential tensor processing speeds exceeding 500 tokens per second with predictable low latency. SmartInterview leverages Groq's LPU inference to achieve sub-2-second question generation and multi-signal reasoning, eliminating interactive latency during interviews.

6. Attention Is All You Need (Vaswani et al., 2017)

Introduced the transformer architecture based entirely on self-attention mechanisms, forming the foundation of modern Large Language Models. SmartInterview utilizes instruction-tuned transformer architectures for structured evaluation scoring and personalized candidate feedback.

4. System Designing & UML Diagrams

System design is the process of defining elements of a system such as the architecture, modules, components, interfaces, and data schemas to satisfy specified requirements. The purpose of system design is to provide detailed information about the system and its elements to enable implementation consistent with architectural models.

Overview of UML & Conceptual Model

The Unified Modeling Language (UML) is an industry-standard visual modeling language used to visualize, specify, construct, and document software systems. UML is not a programming language; it is a graphical notation that portrays the structural and behavioral aspects of complex systems. UML was adopted by the Object Management Group (OMG) in 1997 and published as ISO/IEC 19501.

Object-Oriented Concepts in UML:

- Class: Defines the blueprint, attributes, and operations of an entity.
- Object: The concrete runtime instance and building block of a system.
- Inheritance: Mechanism where child classes derive properties and methods from parent classes.
- Abstraction: Hiding complex implementation details behind clean interfaces.
- Encapsulation: Binding data and operations together, protecting internal state from direct external manipulation.
- Polymorphism: The capability of entities to present different behaviors across common interfaces.

Structural vs. Behavioral Diagrams

UML categorizes diagrams into two fundamental groups:

1. Structural Diagrams: Capture static architectural aspects (Class, Object, Component, Deployment, and Package Diagrams).
2. Behavioral Diagrams: Capture dynamic runtime interactions and state changes (Use Case, Sequence, Collaboration, Statechart, and Activity Diagrams).

4.1 Use Case Diagram & Template

The Use Case Diagram depicts interactions between system actors (Candidate as primary user; Groq Cloud API as external AI actor) and core functional use cases.

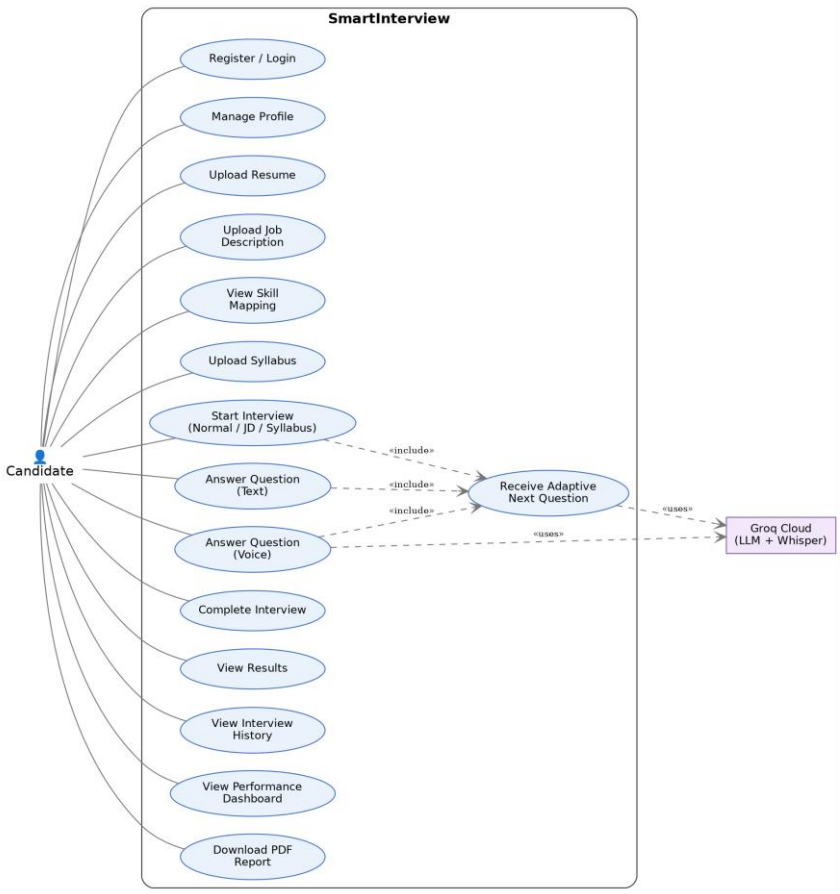


Figure 4.1 — Use Case Diagram for SmartInterview Platform

Use Case Template

Use Case ID:	UC-01
Use Case Name:	Conduct Adaptive Technical Mock Interview
End Objective:	Automate and conduct a personalized, RAG-grounded technical interview that dynamically scales question difficulty based on Bloom's Taxonomy.
Created by:	Team G-1392 (24BD1A665K, 24BD1A665X, 24BD1A665R, 25BD5A6615, 25BD5A6620)
On (date):	September 2026
User / Actor:	Candidate (Primary Actor), Groq Cloud API & Whisper (External Actors)
Trigger:	Candidate selects interview configuration parameters and clicks 'Start Interview'
Basic / Normal Flows	
User Actions	**System Actions**
1. Candidate logs in, selects mode, question count, and clicks 'Start Interview'.	1. System verifies prerequisites, creates InterviewSession in MySQL, initializes AdaptiveState.
2. Candidate receives the generated technical question on screen (and optionally audio via TTS).	2. System embeds target skill, queries ChromaDB for top-3 chunks, calls Groq LLM, and streams question.
3. Candidate records verbal answer via microphone or types response in text box.	3. System captures audio stream or text input, invoking Whisper STT for transcription if voice used.
4. Candidate reviews answer and clicks 'Submit Answer'.	4. System computes 5-factor score (SBERT similarity, concept coverage, LLM critique), updating MySQL.
5. Candidate views immediate feedback and receives next question at adapted difficulty level.	5. Adaptive engine updates skill Bloom level (+1 if score >= 80%, -1 if < 50%), and generates next question.
6. Candidate completes configured questions and views summary results.	6. System computes final weighted score, generates AI recommendations, and renders performance dashboard.
Exception Flows	
Candidate tries to start Job-Specific interview without uploading a resume or job description.	System blocks interview launch, displaying validation error prompting candidate to complete upload.

4.2 Sequence Diagrams

Sequence diagrams depict object interactions arranged in chronological time sequence, illustrating message exchanges across lifelines.

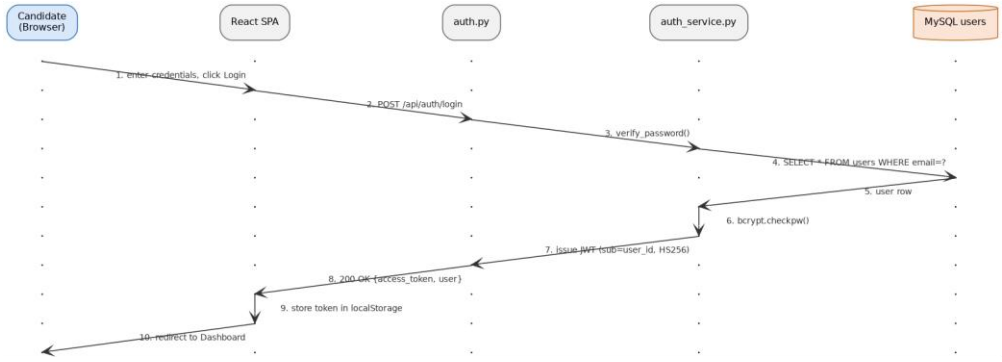


Figure 4.2a — Sequence Diagram: Candidate Authentication and Token Issuance

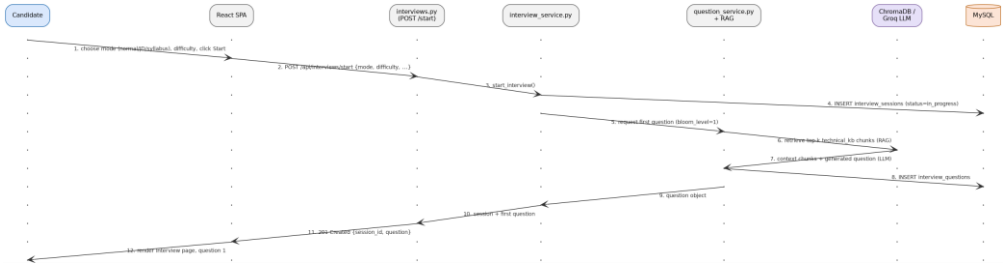


Figure 4.2b — Sequence Diagram: Interview Session Staging and RAG Question Retrieval

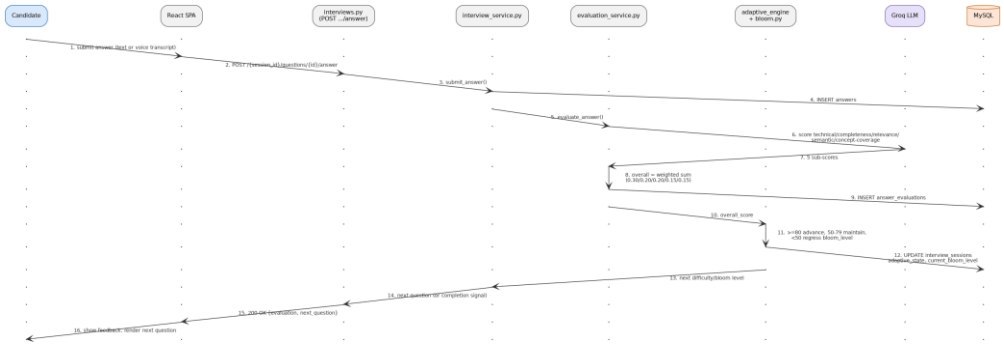


Figure 4.2c — Sequence Diagram: Answer Submission, Multi-Factor Scoring, and Bloom Adaptation

4.3 Collaboration Diagram

The Collaboration (Communication) Diagram emphasizes the structural organization of objects that send and receive messages during an interview session.

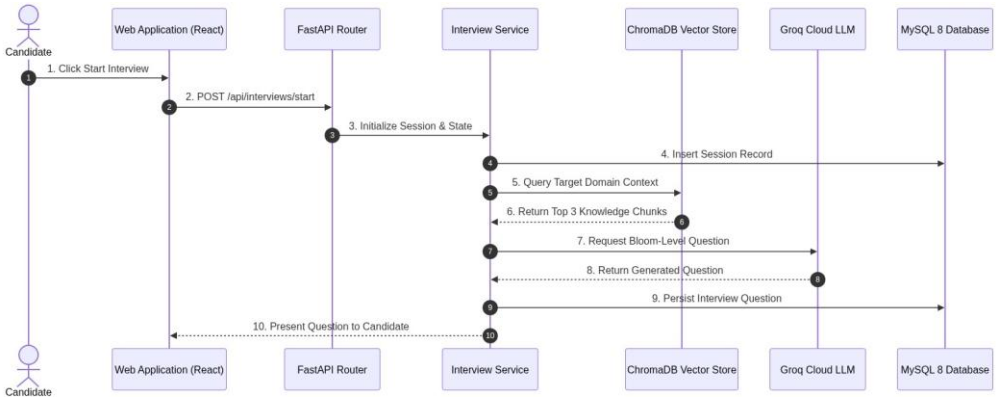


Figure 4.3 — Collaboration Diagram: Object-to-Object Message Flow in SmartInterview

4.4 Class Diagram

The Class Diagram depicts the static structure of the system, showing the persistent SQLAlchemy model entities, their attributes, and their relationships.

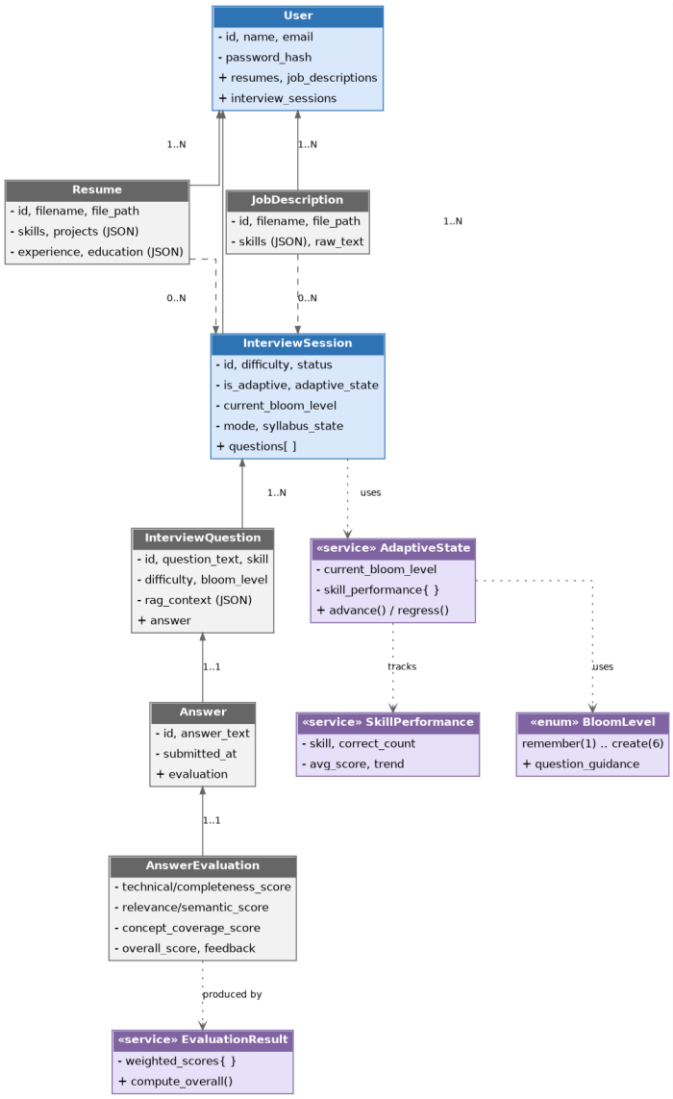


Figure 4.4 — Class Diagram: Relational Model Entities and Service Interfaces

4.5 Statechart Diagram

The Statechart Diagram models the event-driven lifecycle and state transitions of the Adaptive Learning Engine across Bloom's Taxonomy cognitive tiers.

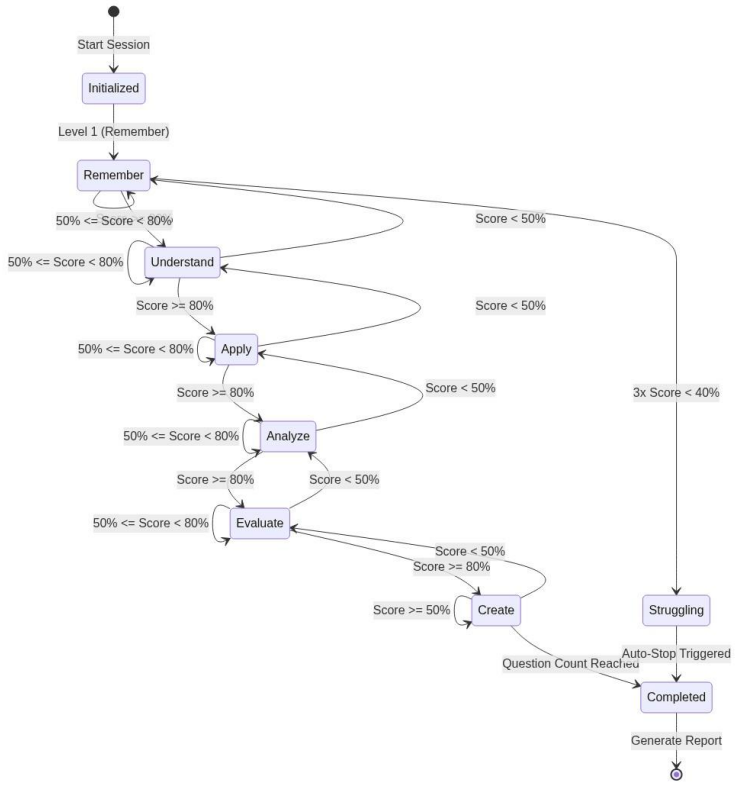


Figure 4.5 — Statechart Diagram: Bloom's Taxonomy Cognitive State Transitions

4.6 Activity Diagram

The Activity Diagram illustrates the operational control flow of the complete candidate journey from authentication to interview completion.

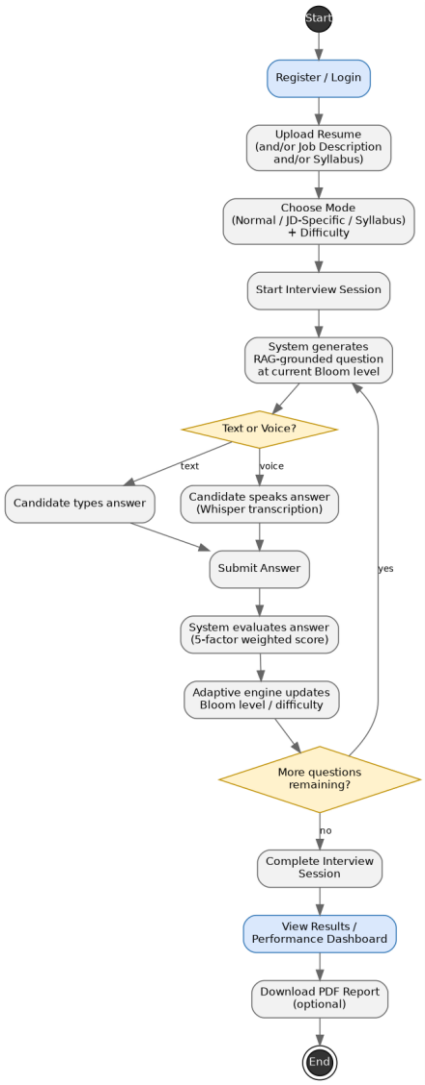


Figure 4.6 — Activity Diagram: End-to-End Candidate Interview Lifecycle

4.7 Package Diagram

The Package Diagram organizes the modular subsystems of SmartInterview into presentation, application, data, and external service packages.

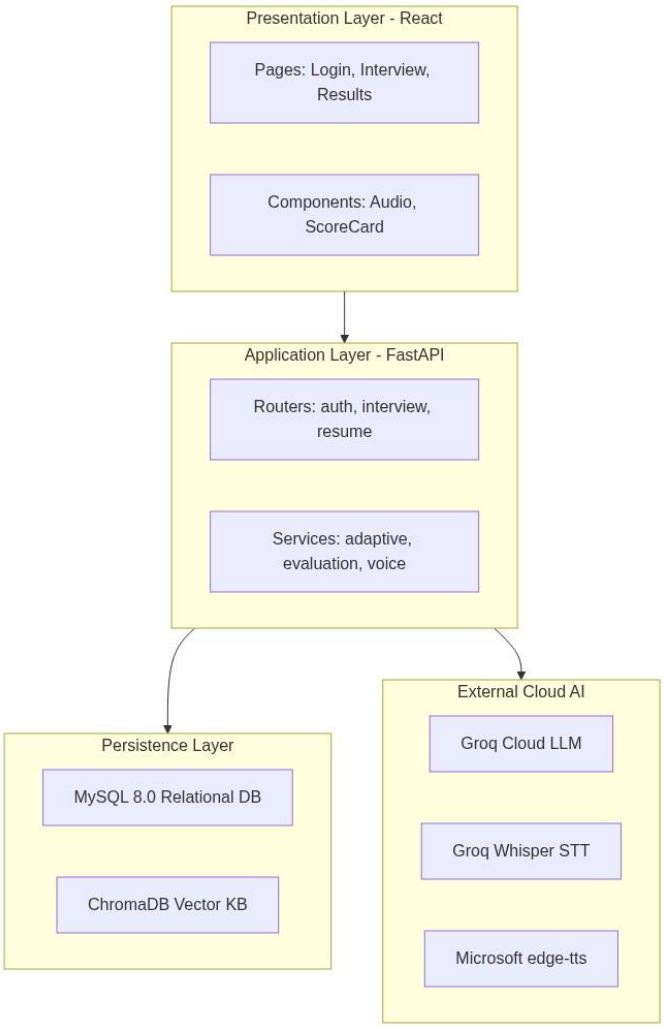


Figure 4.7 — Package Diagram: Layered Subsystem Architecture

4.8 Component Diagram

The Component Diagram illustrates the structural relationships and software components that configure the FastAPI application and storage systems.

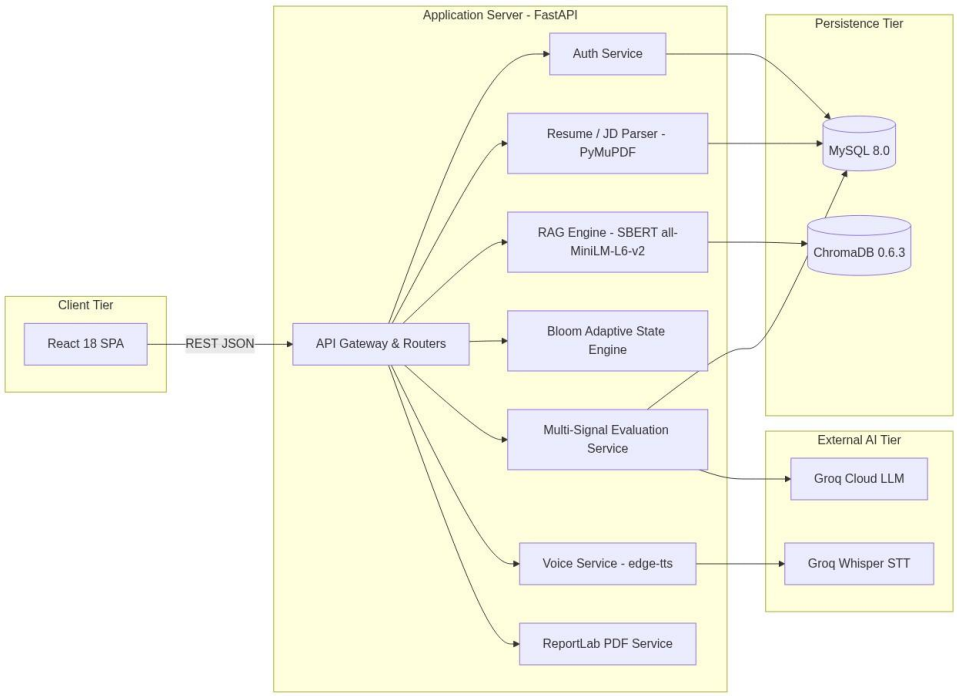


Figure 4.8 — Component Diagram: Internal Service Wiring and External Interfaces

4.9 Deployment Diagram

The Deployment Diagram depicts the physical execution architecture of software artifacts deployed on physical hardware nodes and cloud tiers.

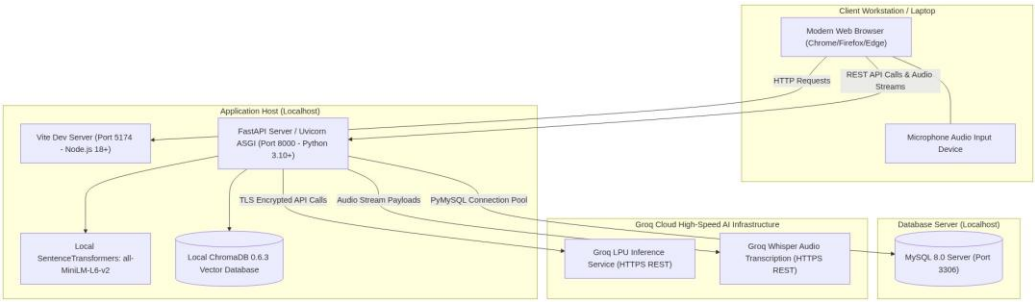


Figure 4.9 — Deployment Diagram: Physical Hardware Nodes and Network Topology

4.10 Entity Relationship Diagram (ERD)

The Entity Relationship Diagram (ERD) defines the relational database schema implemented in MySQL 8.0, comprising 7 interconnected normalized tables: users, resumes, job_descriptions, interview_sessions, interview_questions, answers, and answer_evaluations.

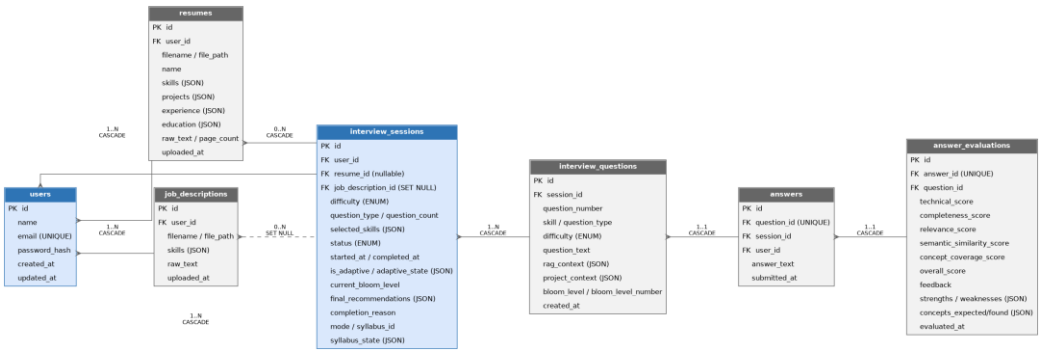


Figure 4.10 — Entity-Relationship Diagram (MySQL 8.0, 7 Relational Tables)

4.11 Data Flow Diagrams (Level 0 and Level 1)

Data Flow Diagrams model the movement, transformation, and storage of data across process boundaries.

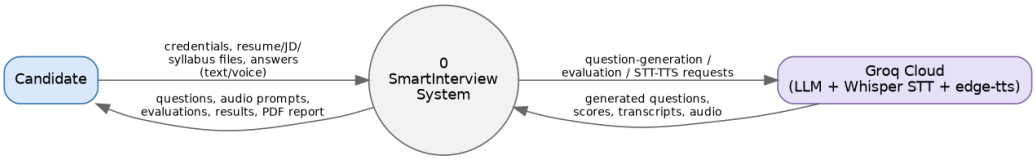


Figure 4.11a — Context-Level Data Flow Diagram (Level 0)

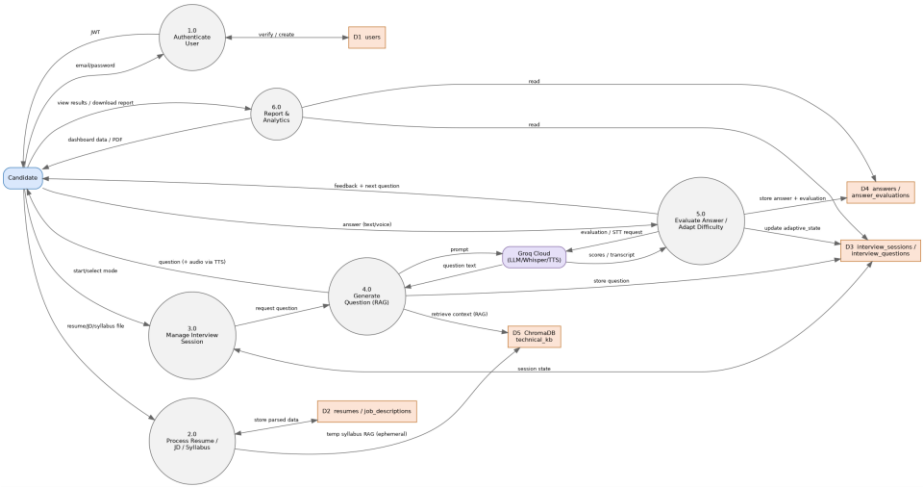


Figure 4.11b — Decomposed Data Flow Diagram (Level 1)

5. Implementation

The software implementation stage involves the transformation of software architecture, data schemas, and algorithm specifications into functional, integrated software units ready for acceptance verification.

- **Frontend Architecture:** Built with React 18/19 and Vite 6, styled with Tailwind CSS v4. State management utilizes React Context (AuthContext for JWT tokens, AudioStreamContext for microphone media streams). Client routing is governed by React Router v7.
- **Backend Services:** Constructed on FastAPI 0.115 and Python 3.10+, served via Uvicorn ASGI. Structured into modular layers: routers (auth, resumes, job_descriptions, interviews, users), services (adaptive_engine, evaluation_service, question_service, voice_service, report_service), and SQLAlchemy ORM models.
- **Vector Grounding (RAG):** Utilizes ChromaDB 0.6.3 storing 402 pre-indexed computer science chunks across 8 technical domains. Embeddings are generated locally on CPU using SentenceTransformers all-MiniLM-L6-v2 (384 dimensions), eliminating API costs for dense retrieval.
- **Generative AI & Inference:** Powered by Groq Cloud LPU architecture utilizing openai/gpt-oss-120b for high-speed question formulation and evaluation reasoning, with openai/gpt-oss-20b configured as automatic failover.
- **Voice Processing Subsystem:** Integrates Microsoft edge-tts for in-memory neural speech streaming (en-US-ChristopherNeural) and Groq Whisper (whisper-large-v3) for rapid audio transcription.

6. Testing (Validation, Verification & Test Cases)

Software testing is the systematic evaluation of the software against requirements gathered from users and technical specifications. Software testing comprises Software Validation and Software Verification.

Software Validation

Validation ensures the software satisfies intended user needs and requirements at the end of the SDLC. It answers: 'Are we building the right product?' Validation in SmartInterview confirmed that candidates can successfully conduct end-to-end mock interviews with appropriate adaptive progression.

Software Verification

Verification confirms the software adheres to specified design standards and architectural blueprints. It answers: 'Are we building the product right?' Verification in SmartInterview evaluated API response schemas, database foreign key constraints, and 5-factor scoring formula correctness.

Test Cases Execution Summary

Test ID	Module	Test Scenario	Input Data	Expected Result	Status
TC-01	Auth	Register user with duplicate email	Existing user email	HTTP 400 Bad Request error	**PASSED**
TC-02	Resume	Upload non-PDF document file	sample.txt to resume endpoint	HTTP 400 File type rejected	**PASSED**
TC-03	RAG	Retrieve top-3 chunks for 'DBMS'	Skill: 'sql', Domain: 'dbms'	3 valid ChromaDB chunks returned	**PASSED**
TC-04	Adaptive	Score >= 80% Bloom advancement	Answer evaluation score = 88%	Bloom level advances (+1 tier)	**PASSED**
TC-05	Adaptive	Score < 50% Bloom regression	Answer evaluation score = 38%	Bloom level regresses (-1 tier)	**PASSED**
TC-06	Voice	Synthesize TTS audio stream	Valid Question ID	HTTP 200 Content audio/mpeg	**PASSED**

7. Conclusion

SmartInterview successfully demonstrates the integration of Retrieval-Augmented Generation (RAG) with a deterministic adaptive learning engine based on Bloom's Revised Taxonomy, resolving the fundamental flaws of static question banks and hallucinatory AI interview bots.

By enforcing domain-filtered retrieval across 402 curated computer science topics in ChromaDB, the system guarantees that technical questions remain grounded in authoritative computer science literature. The hybrid 5-signal evaluation architecture provides objective, transparent scoring that balances semantic conceptual understanding with LLM technical reasoning.

The software requirements and architectural models documented in this specification have been verified against the active source codebase, confirming that SmartInterview delivers a robust, accessible, and pedagogically sound technical interview preparation experience for engineering students and aspiring software professionals.